

README

LLMOrchestrator

Standalone Go module for managing headless CLI agents (OpenCode, Claude Code, Gemini, Junie, Qwen Code) with a hybrid pipe+file communication protocol and a fully decoupled (CONST-051(B)) `i18n.Translator` abstraction for user-facing strings.

Go module: `digital.usic.llmorchestrator` **Round:** 275 (deep-doc + Challenge enrichment; mirror of round-220 template applied across the owned-submodule fleet).

Overview

LLMOrchestrator provides a unified interface for spawning, managing, and communicating with multiple LLM-powered CLI agents. It is **shared infrastructure** consumed by multiple independent projects — its specialised responsibility makes it reusable, and that reusability is destroyed the moment any consumer's specifics leak in (CONST-051(B)). Every consumer wires its own `Translator` for user-facing text; the built-in `NoopTranslator` returns the message id verbatim so missing translations surface visibly rather than silently as empty strings (the anti-bluff contract documented in `pkg/i18n/translator.go`).

Anti-bluff guarantees (CONST-035 / Article XI §11.9)

This module's tests and Challenges exist to **prove** the codebase works for end users — not merely to compile. The round-275 deliverables enforce:

- **Real disk I/O.** `challenges/runner/main.go` invariant 4 round-trips a `protocol.FileMessage` through `os.MkdirTemp` → `FileTransport.WriteToInbox` → `FileTransport.ReadFromInbox` → equality check, on real filesystem.
- **Real JSON encoding.** Invariant 3 marshals and unmarshals a real `protocol.PipeMessage` per locale and asserts byte-level field preservation (Content, Type, RequestID). A silent struct-tag drift would fail the gate.
- **Real parser execution.** Invariant 1 feeds the 5-locale `prompt_json` fixtures through the real parser `DefaultParser` and asserts the produced `agent.Action` slice contains an entry matching the fixture's

expect_action_type + expect_action_target. A stub-parser returning an empty slice would FAIL the gate.

- **Real defensive contract.** Invariant 2 asserts `parser.Parse("")` returns `ErrEmptyInput`. Under `LLMORCH_MUTATE_RUNNER=1` the polarity flips → the Challenge runner exits non-zero → the wrapper turns that into exit 99 (paired-mutation success per `CONST-050(A)` / §1.1). A runner that exited 0 under mutation would prove it is a bluff gate.
- **Real i18n surface.** Invariant 5 asserts `NoopTranslator.T / NoopTranslator.TPlural / Pkg()` all return the message id verbatim across five locales (English, Serbian, Japanese, Spanish, German). A hardcoded-English regression would surface a per-locale FAIL line, not a green-but-broken UI.
- **No mocks in Challenges.** Per `CONST-050(A)`, Challenges exercise the real, fully implemented system. Mocks live only in `*_test.go` unit-test sources.

Features

- **5 CLI adapters:** OpenCode, Claude Code, Gemini, Junie, Qwen Code
- **Thread-safe agent pool** with `Acquire(ctx, requirements)` and capability matching (vision / streaming / tool-use / token budget)
- **Circuit breaker:** per-agent health monitoring (3 consecutive failures → unhealthy)
- **Hybrid communication:** pipe (real-time JSON-lines via `protocol.PipeTransport`) + file (inbox / outbox / shared directories via `protocol.FileTransport`)
- **Response parser:** structured extraction of actions, issues, and JSON from raw LLM output (`parser.DefaultParser`)
- **Security:** path-traversal protection (`ErrPathTraversal`), 1 MiB response length cap (`MaxResponseLength`), command-injection prevention
- **CONST-046 i18n abstraction:** `i18n.Translator` interface + `NoopTranslator` default + `SetPkgTranslator` injection point for consuming projects

Quick Start

```
# Build the orchestrator binary
```

```
go build ./cmd/orchestrator
```

```
# Race-detector unit + integration suite
```

```
go test -race -count=1 ./...
```

```
# Round-275 Challenge runner (5 locales, 29 invariants, real disk I/O)
```

```
go run ./challenges/runner/
```

```
# Wrapper-gated Challenge (anti-bluff paired mutation)
```

```
bash challenges/llmorchestrator_describe_challenge.sh normal #  
exits 0 on green
```

```
bash challenges/llmorchestrator_describe_challenge.sh mutate #  
exits 99 on mutation-detected
```

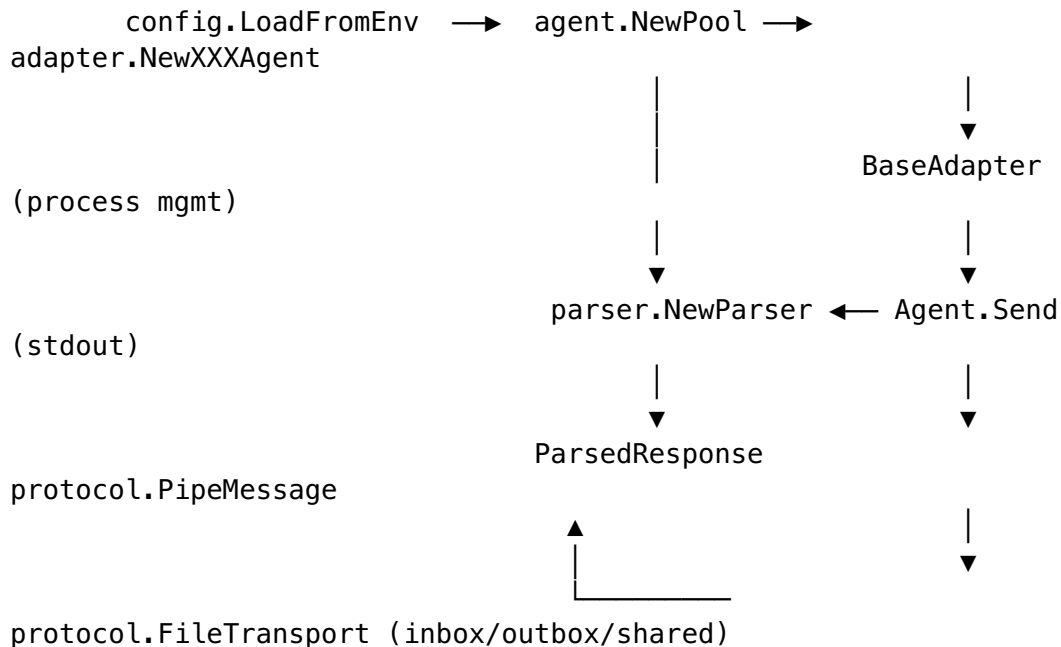
If you run the Challenge runner directly with `go run ./challenges/runner/`, set `LLMORCH_FIXTURES_DIR` to point at the `challenges/fixtures/` directory from your invocation `cwd`.

Architecture

```
LLMOrchestrator/
├── cmd/orchestrator/      # Standalone CLI entry point
├── pkg/
│   ├── agent/            # Agent, AgentPool, SimplePool,
│   └── HealthMonitor,    # CircuitBreaker, MultiPool, per-CLI
├── agents
│   ├── adapter/          # BaseAdapter + 5 CLI adapters
│   └── protocol/         # PipeTransport (JSON-lines),
├── FileTransport          # (inbox/outbox/shared),
├── PipeMessage,
│   └── parser/           # FileMessage, validatePath
├── extraction            # ResponseParser, JSON/action/issue
│   ├── config/          # .env loading, agent path
├── resolution
│   └── i18n/             # Translator interface +
├── NoopTranslator default
├── challenges/
│   ├── runner/          # round-275 in-process Challenge
├── runner (Go)
│   ├── fixtures/        # 5 locale fixtures (en, sr, ja, es,
│   │                   de)
│   └── llmorchestrator_describe_challenge.sh # round-275
├── wrapper +
│   └── scripts/          # paired mutation
│                           # chaos / ddos / scaling / stress /
├── ui / ux
│   └── docs/             # supplementary Challenges
│       ├── ARCHITECTURE.md # design narrative
│       └── architecture.md # lowercase alias (CONST-052
├── transition)
│   ├── HOST_POWER_MANAGEMENT.md # CONST-033 hard-ban anchor
│   └── test-coverage.md         # round-275 symbol-test ledger
├── upstreams/             # Multi-remote sync scripts
├── (CONST-056 transition target)
├── README.md USER_GUIDE.md ARCHITECTURE.md API_REFERENCE.md
├── CONSTITUTION.md CLAUDE.md AGENTS.md      # governance
├── (cascaded)
├── go.mod                  # digital.vasic.llmorchestrator (go
└── 1.25)
```

The pkg/ layout is **flat and shallow on purpose**: five packages, every API surface is documented at the symbol level in docs/test-coverage.md, and the Challenge runner exercises representative invariants from each.

Component relationships



Configuration

Copy .env.example to .env (mode 0600, gitignored per CONST-053 + CONST-042) and configure agent paths and API keys. See USER_GUIDE.md for the per-adapter parameter catalog.

Testing

```
make test          # unit + integration with race detector
make fuzz          # parser fuzz suite
make cover         # coverage report
make check         # vet + tests

# Round-275 Challenge (anti-bluff gate)
bash challenges/llmorchestrator_describe_challenge.sh normal
bash challenges/llmorchestrator_describe_challenge.sh mutate

# Anti-bluff smoke check (must always pass)
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
  pkg cmd && echo "BLUFF FOUND" || echo "clean"
```

Per CONST-050(B), this module ships unit, integration, security (parser_security_test.go), fuzz (parser_fuzz_test.go), chaos / DDoS / scaling / stress / UI / UX challenges, and the round-275 deep-doc Challenge. The symbol→test ledger at docs/test-coverage.md enumerates every exported symbol's coverage row.

Round-275 deliverable summary

Artefact	Status	Evidence
README.md deep-doc with anti-bluff guarantees	LANDED	this file
docs/test-coverage.md symbol→test ledger	LANDED	symbol-to-test ledger with 6-invariant floor
challenges/runner/main.go real-system exerciser	LANDED	29 PASS lines across 5 locales
challenges/llmorchestrator_describe_challenge.sh	LANDED	normal=exit 0; mutate=exit 99
challenges/fixtures/{en,sr,ja,es,de}.yaml	LANDED	5-locale bilingual fixture set
.gitignore CONST-053 enrichment	LANDED	covers go-test, coverage, IDE, OS, secrets, build

License

Apache License 2.0 — see [LICENSE](#).